API_REFERENCE

API Reference

Package: pkg/loader

Types

Document

```go
type Document struct {
    Path       string
    Format     string
    Title      string
    Content    string
    Sections   []Section
    Links      []string
    ModifiedAt time.Time
}
```

Section

```go
type Section struct {
    Title   string
    Level   int
    Content string
    Line    int
}
```

Interfaces

Loader

```go
type Loader interface {
    LoadDir(ctx context.Context, path string) ([]Document, error)
    LoadFile(ctx context.Context, path string) (Document, error)
    SupportedFormats() []string
}
```

Functions

- `NewDefaultLoader(formats []string) *DefaultLoader` -- Create a loader for the given file extensions.

Constants

- `MaxFileSize = 10 * 1024 * 1024` -- Maximum file size (10 MB).

Package: pkg/feature

Types

`Feature`

```go
type Feature struct {
    ID, Name, Description string
    Category              FeatureCategory
    Platforms             []string
    Priority              string
    Screens               []string
    TestSteps             []TestStep
    SourceDoc, SourceSection string
}
```

`FeatureMap`

```go
type FeatureMap struct {
    Features       []Feature
    Screens        []ExpectedScreen
    Workflows      []Workflow
    Categories     map[FeatureCategory][]Feature
    PlatformMatrix map[string][]Feature
    DocGraph       *docgraph.DocGraph
    GeneratedAt    time.Time
    ProjectRoot    string
}
```

`FeatureCategory`

Constants: `CategoryFormat`, `CategoryUI`, `CategoryNetwork`, `CategorySettings`, `CategoryStorage`, `CategoryAuth`, `CategoryEditor`, `CategoryOther`.

Interfaces

`FeatureMapBuilder`

```
type FeatureMapBuilder interface {
    BuildFromDocs(ctx context.Context, docs []loader.Document)
        (*FeatureMap, error)
    Enrich(ctx context.Context, fm *FeatureMap, agent
        llm.LLMAgent) error
    Merge(maps ...*FeatureMap) *FeatureMap
}
```

Functions

- `NewBuilder(projectRoot string) *DefaultBuilder`
- `NewFeatureMap(projectRoot string) *FeatureMap`
- `GenerateID(name string) string` -- Deterministic feature ID generation.
- `AllCategories() []FeatureCategory`
- `ValidCategory(s string) bool`

Methods on FeatureMap

- `AddFeature(f Feature)`
- `FeatureByID(id string) *Feature`
- `FeaturesForPlatform(platform string) []Feature`
- `FeaturesForCategory(category FeatureCategory) []Feature`

Package: pkg/coverage

Types

`CoverageReport`

```
type CoverageReport struct {
    Total, Verified, Failed, Skipped, Unverified int
    OverallPct float64
    ByPlatform map[string]float64
    ByCategory map[string]float64
    Issues     []Issue
}
```

`Evidence`

```
type Evidence struct {
    ScreenshotPath, VideoPath, LogPath, Description string
    VideoOffset time.Duration
```

```
    Timestamp   time.Time
}
```

Issue

```go
type Issue struct {
    Type, Severity, Title, Description, ScreenID string
    Evidence []string
}
```

VerificationState

Constants: `StateUnverified`, `StateVerified`, `StateFailed`, `StateSkipped`.

Interfaces

CoverageTracker

```go
type CoverageTracker interface {
    RegisterFeature(f Feature, platforms []string)
    MarkVerified(featureID, platform string, evidence Evidence)
    MarkFailed(featureID, platform string, issue Issue)
    MarkSkipped(featureID, platform string, reason string)
    Coverage() CoverageReport
    CoverageByPlatform(platform string) float64
    CoverageByCategory(category string) float64
    Unverified() []Feature
    Export() CoverageSnapshot
}
```

Functions

- `NewTracker() CoverageTracker`

Package: pkg/docgraph

Types

Node

```go
type Node struct {
    ID    string `json:"id"`
    Title string `json:"title"`
}
```

Edge

```go
type Edge struct {
    From string `json:"from"`
    To   string `json:"to"`
}
```

Functions

- `New() *DocGraph`
- `ImportJSON(data []byte) (*DocGraph, error)`

Methods on DocGraph

- `AddNode(id, title string)`
- `AddEdge(from, to string)`
- `Nodes() []Node`, `Edges() []Edge`
- `NodeCount() int`, `EdgeCount() int`
- `HasNode(id string) bool`, `HasEdge(from, to string) bool`
- `Neighbors(id string) []string`, `IncomingEdges(id string) []string`
- `Export() GraphSnapshot`
- `ExportJSON() ([]byte, error)`
- `ExportMermaid() string`

Package: pkg/llm

Interfaces

LLMAgent

```go
type LLMAgent interface {
    Summarize(ctx context.Context, text string) (string, error)
    ExtractFeatures(ctx context.Context, text string)
        ([]RawFeature, error)
    ClassifyFeature(ctx context.Context, feature RawFeature)
        (FeatureCategory, error)
    InferScreens(ctx context.Context, features []Feature)
        ([]ExpectedScreen, error)
    GenerateTestSteps(ctx context.Context, feature Feature)
        ([]TestStep, error)
}
```

Functions (Prompt Templates)

- `SummarizePrompt(text string) string`
- `ExtractFeaturesPrompt(text string) string`

- `ClassifyFeaturePrompt(feature RawFeature) string`
- `InferScreensPrompt(features []Feature) string`
- `GenerateTestStepsPrompt(feature Feature) string`

Constants

- `PromptVersion = "1.0.0"`

Package: pkg/config

Types

`Config`

```go
type Config struct {
    DocsRoot     string
    AutoDiscover bool
    Formats      []string
}
```

Functions

- `DefaultConfig() *Config`
- `LoadFromEnv(path string) (*Config, error)`
- `LoadFromMap(env map[string]string) *Config`